



Module 27-1

Introduction to UVM Methodology

cādence®

This page does not contain notes.

Introduction to UVM Objectives

In this module, you

- Discuss the benefits of the Universal Verification Methodology (UVM)
- Define the architecture of a UVM Verification Component (UVC)
- Explain the differences between interface and module UVCs

Warning – this is a very simple introduction to UVM using an incomplete test environment

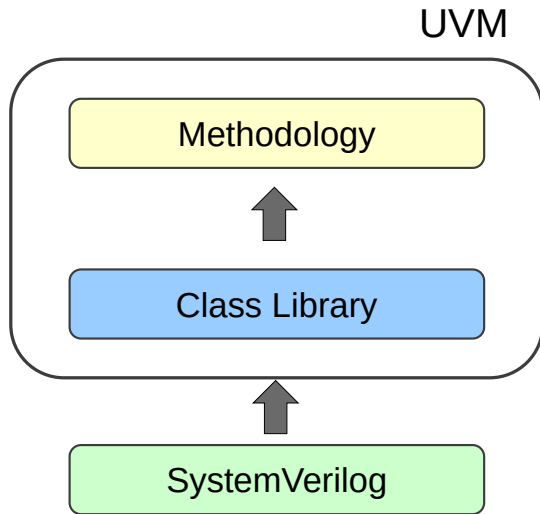
Please see the full training class for much more information

- SystemVerilog Accelerated Verification using UVM
 - Builds a complete UVM test environment from scratch



This page does not contain notes.

What Is UVM?



- A class library of verification building blocks
 - Written in standard IEEE1800 SystemVerilog
- A proven verification methodology
 - Defines how to use the class library
 - Scalable from block-level to system-level verification
- IEEE standard 1800.2
 - Documented and maintained by IEEE
 - Open source reference library from Accellera
 - Apache license
 - Does not lock users into a single vendor solution

3

© Cadence Design Systems, Inc. All rights reserved.



UVM is a library of SystemVerilog classes which implement verification building blocks along with support features such as phasing, messaging, factory etc. The library provides additional class automation, such as print, copy, comparison, cloning etc., which are not available in SystemVerilog.

The library API is defined as an IEEE standard (1800.1) and Accellera (the industry standard body) provides a reference implementation of the library. By extending our testbench environment from the UVM library, we can exploit the built-in automation and support features.

However, UVM is also a methodology for using the building blocks and features of the library. By following the methodology, you can create scalable verification components which can be used from block-level to system-level verification.

By following the methodology and the architecture it dictates, we can create standardized, reusable verification blocks.

UVM has an Apache license, which allows commercial IP to be built and sold on top of UVM.

All this means that companies can create internal, external and commercial verification IP conforming to UVM, and hence allowing the greatest possible reuse.

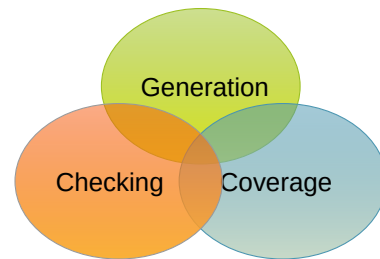
UVM Verification Component (UVC)

Definition

- Verification IP block
- Reusable, pre-verified, configurable, plug-and-play
 - Architected and packaged in a consistent way
- Complete: provides all the logic required for verification
 - Needs to be re-used and configured as a whole

Two types of UVC

- Interface UVCs
 - Bus-based UVCs (for example, PCI and AHB)
 - Communication UVCs (for example, Ethernet, MAC, Datalink)
- Module UVCs (for example, UART core, ALU module)
 - Scoreboard
 - Register model



This page does not contain notes.

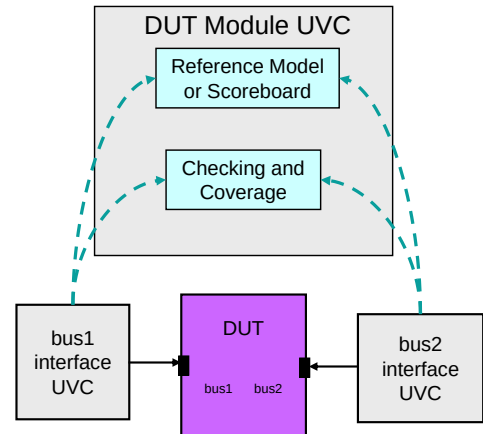
Module and Interface UVCs

Interface UVC

- Specific to a protocol, bus, or set of signals on the DUT
- Generates stimulus for the interface
- Reusable everywhere interface is used
- Set architecture and configuration

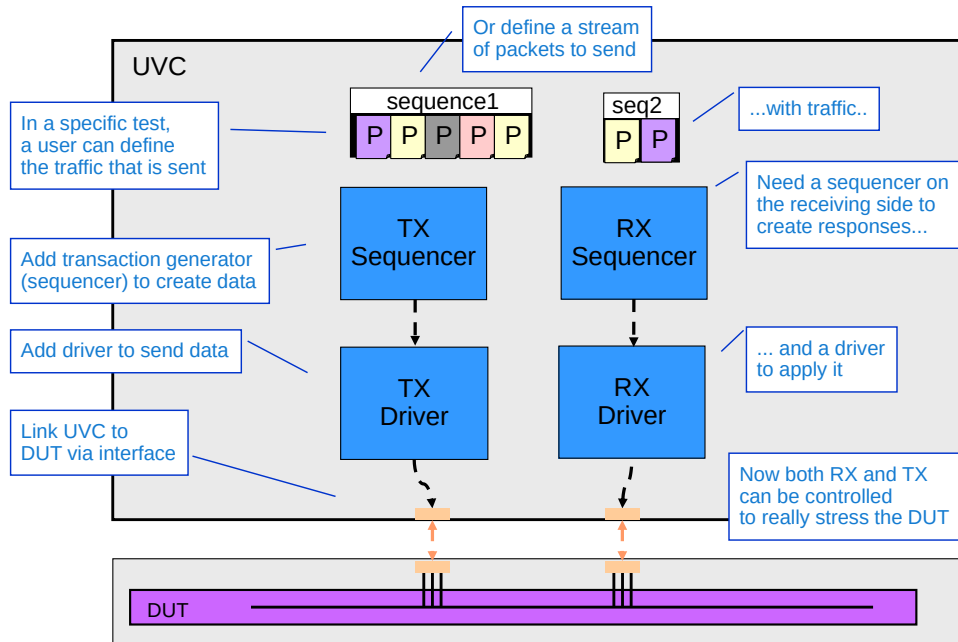
Module UVC

- Specific to DUT or flavor of DUT
- Provides checks and coverage across interface UVCs
- Reusable everywhere DUT is used
- No set architecture



This page does not contain notes.

Creating an Interface UVC



6

© Cadence Design Systems, Inc. All rights reserved.

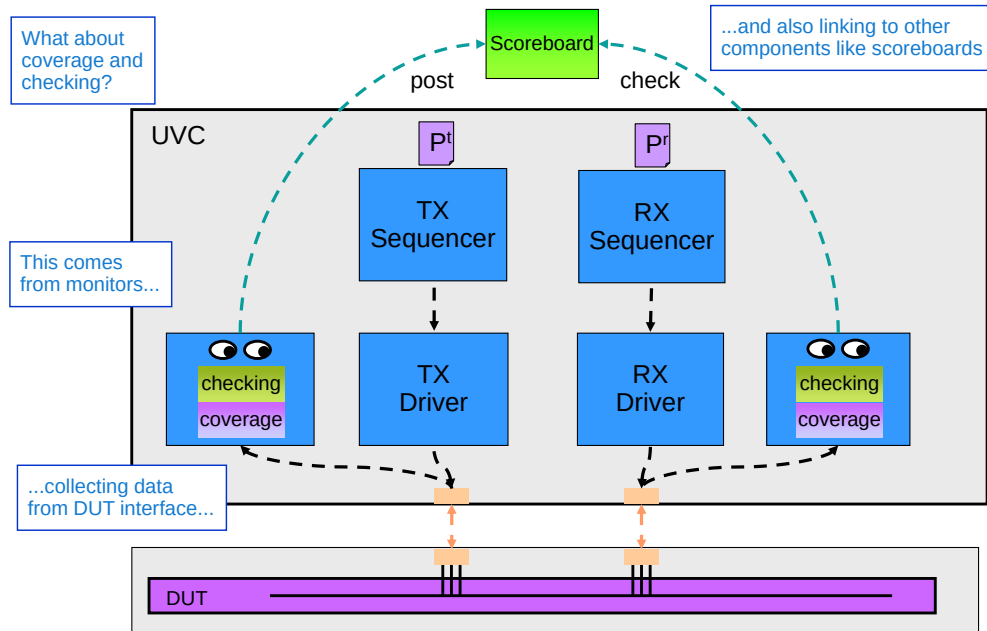
cadence

The TX (transmitter) side sends data into the DUT. The RX (receiver) side receives data from the DUT.

The implication here is that TX/RX components are related, for example, they might send and receive functionality on a single port of a multi-port switch network. If the TX/RX components had different protocols then they would be in different UVCs. If the TX/RX components had the same protocol, but were part of different interfaces to the DUT, they would again be in different UVCs.

For example, a four-port switch with the same protocol for each port, and TX/RX functionality in each port, would have four separate instances of the same UVC, one for each port. The first instance would implement the TX/RX functionality for port 1, the second for port 2, etc.

Monitors, Checking and Coverage



7

© Cadence Design Systems, Inc. All rights reserved.

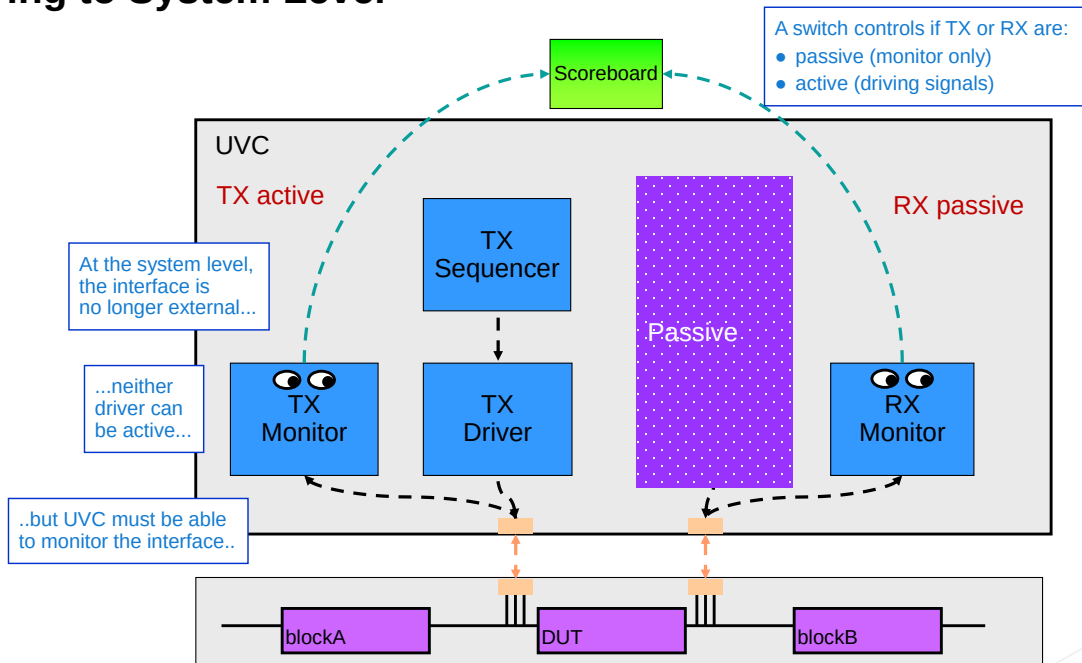
cadence

The sequencer and driver allows us to send stimulus into and capture responses out of the DUT. However we still need to check the outputs are correct for the given inputs and to use coverage to measure what we have and have not tested.

Both coverage and checking should come from monitors that collect information directly from the DUT.

The scoreboard checks the operation of the DUT by collecting input and output data, and checking that the outputs received are compatible with the inputs sent.

Moving to System Level



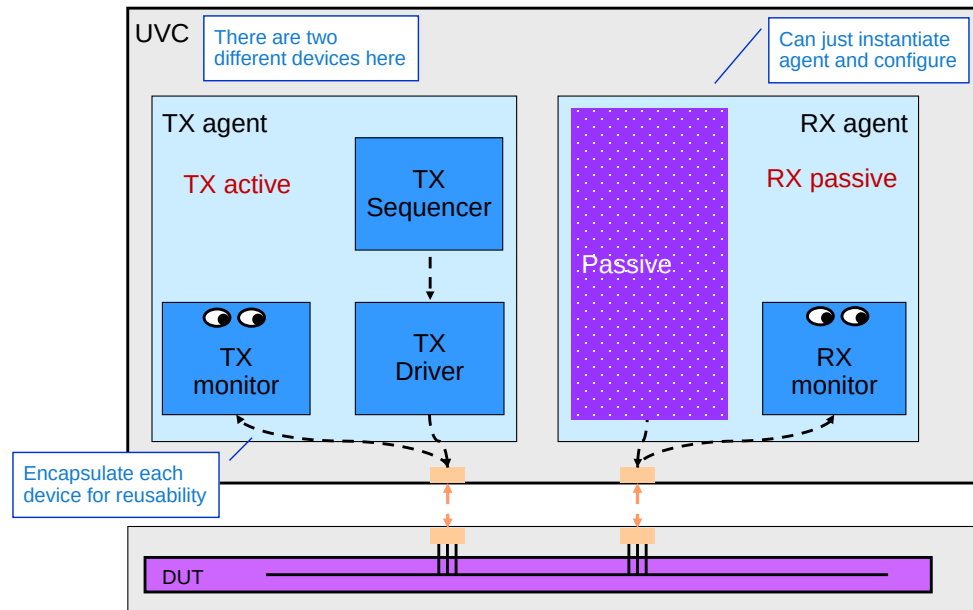
8

© Cadence Design Systems, Inc. All rights reserved.

cadence

At the system level, the interface to the design under test (DUT) is no longer an external interface. Another RTL block drives it, but we still want to have passive checking and coverage on the interface. We need the monitor component to be always be there, whether we drive the DUT signals or another RTL block drives the signals.

Interface UVC Architecture



9

© Cadence Design Systems, Inc. All rights reserved.

cadence

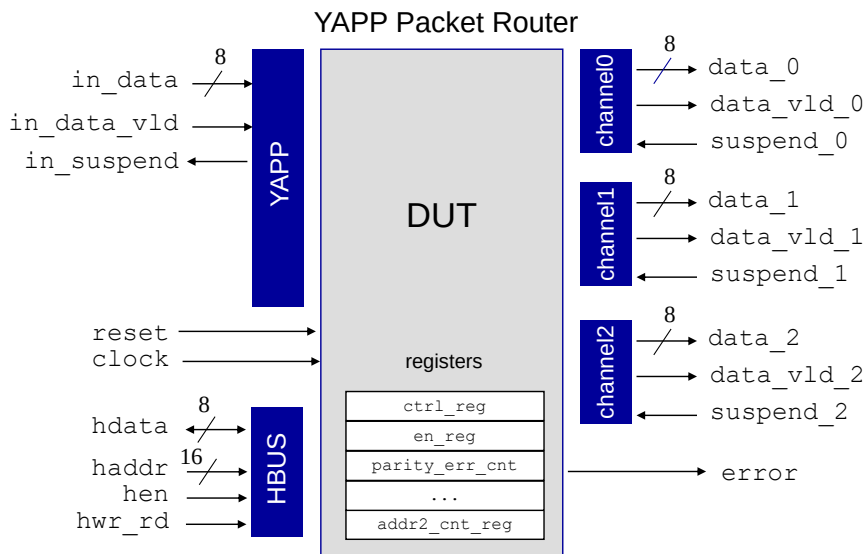
There are two different devices here. Each has a sequencer, driver and monitor, internal communication between them, some configuration, etc.

The device internals are not structured, which might make them harder to reuse and debug. We may also want to instantiate more than two devices (e.g., AHB bus).

Encapsulating each device within a single container will simplify the usage.

Users of the environment do not need to worry about hookup of sub-components. They can just instantiate an agent and configure it for the right task.

DUT Example: YAPP Router



10 © Cadence Design Systems, Inc. All rights reserved.



We will be using the YAPP router as the DUT for both lab exercises and lecture examples throughout this course.

The packet router accepts data packets on a single input YAPP port and routes the packets to one of three output channels: Channel0, Channel1, or Channel2. It has a HBUS interface for programming registers that are described shortly...

error is set when packet is found to have incorrect parity.

A more detailed description of the YAPP router specification is given in your lab book.

Router DUT Packet Specification

A packet is a sequence of bytes:

- First byte is a header
- Next variable set of bytes is the payload
- Last byte is the parity

Header is a 2-bit address field and a 6-bit length field

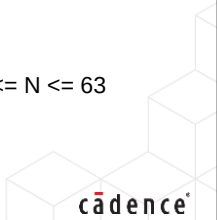
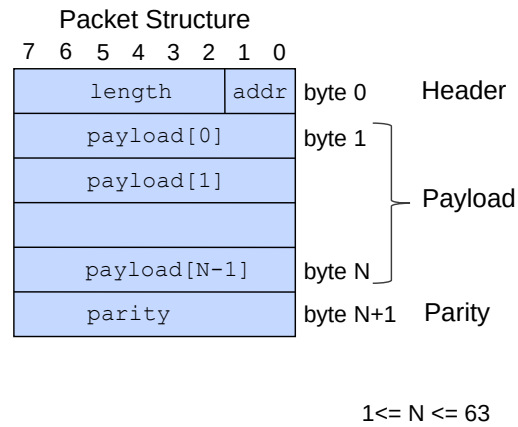
- Address determines to which output channel the packet is routed
 - Address “3” is illegal
- Length specifies the number of bytes in the payload

Payload has a minimum size of 1 byte and a maximum size of 63 bytes

Parity is a full byte of even parity

- Calculated by XORing the header and every byte of the payload

Router raises error output on receiving bad parity



A packet is a sequence of bytes.

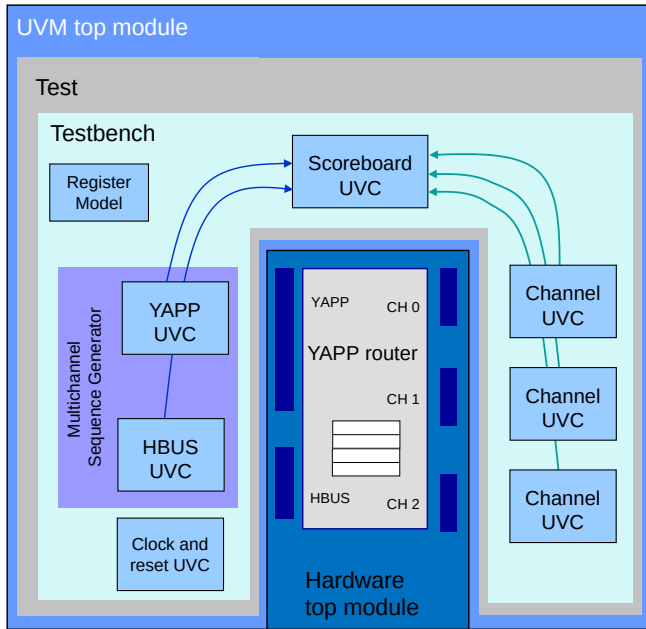
The first byte of the packet is the header. It has two fields:

- 2-bit address field representing the output channel to which the packet should be routed. Valid addresses are 0-2. Address 3 is illegal.
- 6-bit length field representing the number of bytes in the payload (excluding the header and parity).

Bytes 1 to N of the packet is the data payload. Minimum length of the payload is 1 byte. Maximum length is 63. Therefore N and the length field of the header are equal and in the range 1 to 63.

Byte N+1 is the parity byte. Parity is even and calculated over the header and payload bytes. Parity can be calculated by simply XOR'ing the header with every byte of the payload.

How Will We Verify the YAPP?



- Create a UVC for each interface and clock/reset.
- Testbench instantiates and configures UVCs.
- Test instantiates testbench and controls stimulus.
 - Many different tests can be defined for one testbench.
- Add a mechanism to coordinate stimulus from multiple UVCs.
- Add scoreboard for end-to-end checking.
- Add a Register Model to help verify DUT register behavior.
- Hardware top module instantiates DUT and interfaces.
- UVM top module instantiates test and connects UVC interfaces.

12 © Cadence Design Systems, Inc. All rights reserved.



Don't worry about all these components yet... we will work through them and eventually build each piece.

This slide describes the architecture for a UVM verification environment compatible with hardware acceleration. Specifically the creation of separate top level modules for the software side (UVM) and the hardware side.



Module 27-2

Controlling UVC Stimulus with Sequences

cādence®

This page does not contain notes.

Sequences Objectives

In this module, you

- Use UVM macros and methods to create sequences to implement common stimulus requirements:
 - Simple randomized data
 - Directed tests
 - Stimulus controlled by properties
 - Hierarchical stimulus
- Define which sequence is executed by the sequencer



This page does not contain notes.

What Are Sequences?

A sequence is a set of transactions that accomplish a defined task for the DUT.

- Typically, tests are described at the transaction level
 - Easier to write, maintain, and control
 - Examples: instructions, packets
- Single items cannot capture the high-level intention
 - Ordered stream of transactions is required
 - Examples: program generation, connection-based protocols

Some sequences should be provided as part of a reusable component...

- Capture important scenarios that should be exercised
- Provide examples for a test writer to extend from or reuse

...however, a test writer will frequently have to create additional sequences



This page does not contain notes.

UVM Sequence Requirements

- Easy to write and use
 - No need to learn complex mechanism
 - Minimize the protocol/environment knowledge requirements
- Combine declarative and procedural constraints
- Can be generated on-the-fly or at time zero
- Can nest sequences
- Can be timed as well as procedural
 - For acceleration, delays should be mapped to interface methods (see later)
- Ability to create a system-level sequence – drive multiple interfaces from a single sequence (described later)

16 © Cadence Design Systems, Inc. All rights reserved.



UVM sequences need to be easy to use without complex syntax or methodology overheads. They need to be easy to use so that test writer, with only a minimal knowledge of the test environment and the stimulus protocol, can use and create sequences .

Sequences need to be able to combine the default declarative constraints which apply to every data item (e.g., declared in the `yapp_packet` class) with inline procedural constraints to modify individual data items.

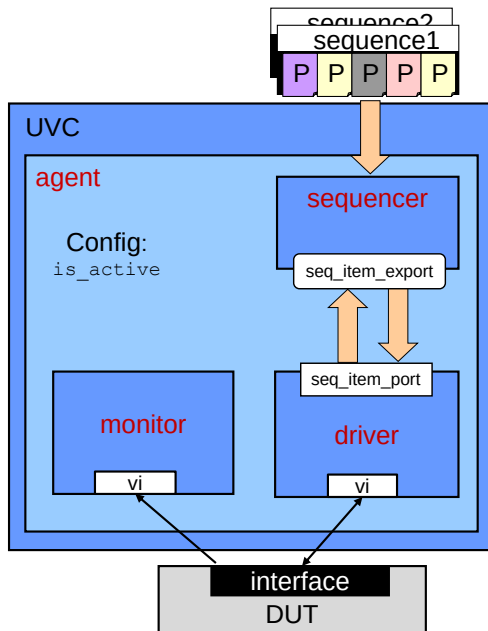
Sequences should support the generation of data items as required during the run phase, perhaps in response to data from the DUT, or the generation and storage of thousands of data items at the beginning of the simulation, which is a technique for hardware acceleration and emulation.

Sequences must support nesting to allow complex sequences to be built out of simple sequence building blocks.

Sequences must support timed behavior, such as delays, event expressions and parallelism (through fork-join blocks) as well as procedural behavior such as looping, conditional branching and iteration. However for acceleration efficiency, explicit delays and event expressions should be executed in the hardware partition only. A sequence should trigger a driver to execute interface methods to, for example, count clocks or wait for DUT events.

Finally we need to create sequences which control more than more UVC, to synchronize stimulus and behavior for all the inputs to the DUT. System-level (or virtual) sequences will be covered later in the class.

Sequences and UVCs Review



Sequence – **data item**

- Defines stimulus pattern of individual items (YAPP packets)
- Can contain declarations, procedural code, timing, constraint blocks, etc.
- Can reuse other predefined sequences
- Provides a standard test writer interface

Sequencer – **component**

- Provides randomization of sequences and items
- Connected to a driver
- Executes sequence by supplying individual items (YAPP packets) to driver on demand

Driver – **component**

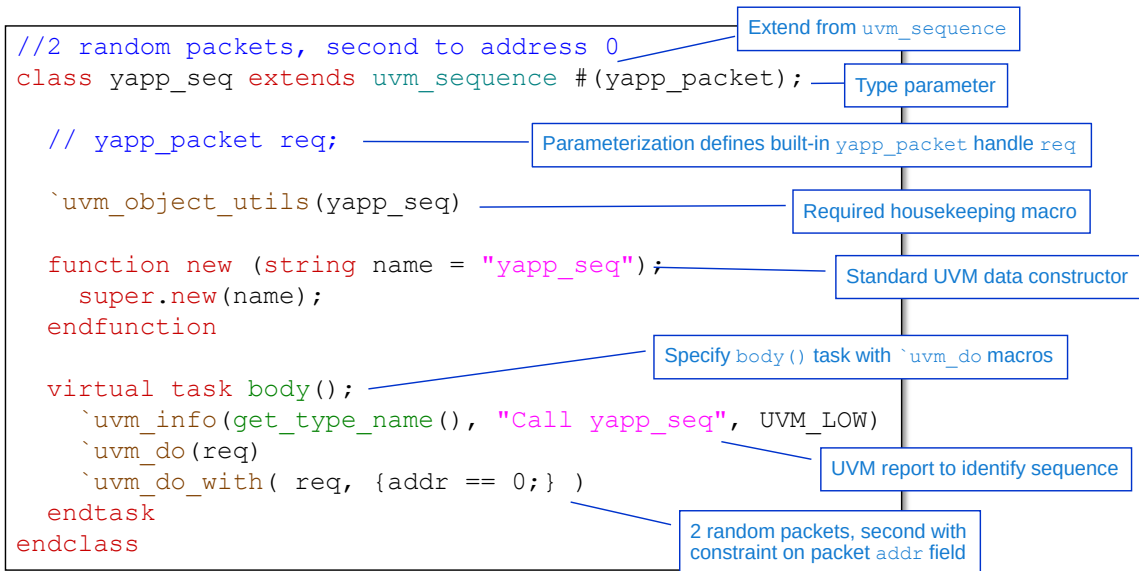
- Connected to a sequencer
- Requests the next item (YAPP packet) from sequencer
- Implements the protocol for driving the item on the interface

17 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

UVM Sequences Example



- Sequences are data items with a UVM data constructor and object utility macro

18 © Cadence Design Systems, Inc. All rights reserved.



The sequence has a type parameter to match the type parameter on its associated sequencer and connected driver. Type parameterization defines a built-in handle on the type named `req`.

The type parameter (`yapp_packet`) must be of a type which derives from `uvm_sequence_item` in order to use `uvm_do` macros.

Remember a sequence is a data item. Therefore it has a data constructor with single `name` argument. A sequence uses the standard object utilities macro to register the sequence with the factory.

The stimulus provided by the sequence is defined in a `body()` task. Inside the task we typically use UVM `do` macro variants to define the stimulus. This sequence creates one random packet with the default `yapp_packet` constraints (equivalent to `req.randomize()`), followed by a second randomized packet with a constraint specifying the `addr` field of the packet equals zero (equivalent to `req.randomize()` with `{addr == 0;}`)

However the `do` macros carry out much more functionality than just randomizing the packet, as we shall see on the following slides.

`uvm_do Macros: What Do They Do?

Step	Summary	Description	Methods
1	creation	Create instance, set parent and link to sequencer	create
2	synchronize	Wait for sequencer request	start_item
3	pre_do hook*	Executes the optional <code>pre_do()</code> task of the sequence	
4	randomization	Randomize the instance. If randomization fails, a warning is issued	randomize
5	mid_do hook*	Executes the optional <code>mid_do()</code> function of the sequence	finish_item
6	send item and wait until done	For items: send item to sequencer, and wait until item done For subsequences: execute the sequence <code>body()</code> method	
7	post_do hook*	Execute the optional <code>post_do()</code> function of the sequence	

* Allows customization of macro behavior, but rarely used

19 © Cadence Design Systems, Inc. All rights reserved.



The `uvm_do` macro performs seven steps to create and randomize an item, and synchronize with the sequencer to pass the item to the driver under the control of the `get_next_item/item_done` method calls.

There are various ways to customize the behavior of the `uvm_do` macro. One (less common) option is to use hook methods (similar to pre/post-randomise). If you define the hook method for the sequence, it is automatically executed at a defined point in the macro execution.

```
virtual task pre_do(bit is_item);
```

- Called after the sequencer has selected this sequence, and before the item is randomized. Although a task, consuming simulation cycles may result in unexpected behavior on the driver.

```
virtual function void mid_do(uvm_sequence_item this_item);
```

- Called after the item has been randomized, and before the item is sent to the driver.

```
virtual function void post_do(uvm_sequence_item this_item);
```

- Called after the sequencer has received the `item_done()` call from the driver.

A more common customization option is to replace `uvm_do` with other macro variants (see next slide) or to use method calls instead (see later). These perform the same actions as `uvm_do`, but allow us to insert custom code between various steps in the flow.

A warning (`uvm_warning`) is issued if randomization fails due to constraint errors, *but* the simulation continues with a non-randomized data item.

Additional `uvm_do Macros

For full flexibility, additional macros can be used for item and sequence execution.

Syntax

- ``uvm_*(variable)`
- ``uvm*_with(variable, { constraint-block})`

<code>`uvm_do</code>	Performs all seven steps from previous slide (creation to <code>post_do</code>)
<code>`uvm_do_with</code>	Same as <code>`uvm_do</code> , but adds in-line constraints during randomization (<code>randomize with {}</code>)
<code>`uvm_create</code>	Only performs step 1 (creation)
<code>`uvm_send</code>	Performs steps 2-7 (synchronize to <code>post_do</code>), but skips step 4 (randomization)
<code>`uvm_rand_send</code>	Performs steps 2-7 (synchronise to <code>post_do</code>)
<code>`uvm_rand_send_with</code>	Performs steps 2-7 (synchronize to <code>post_do</code>) and adds additional constraints for randomization (step 4)



Syntax

- ``uvm_do(variable)`
- ``uvm_do_with(variable, { constraint-block})`
- ``uvm_create(variable)`
- ``uvm_send(variable)`
- ``uvm_rand_send(variable)`
- ``uvm_rand_send_with(variable, { constraint-block})`

The variable must be derived from `uvm_sequence_item` or `uvm_sequence`.

Directed Stimulus with Macros

```
//send illegal packet to address 3
class directed extends uvm_sequence #(yapp_packet);
  int ok;
  `uvm_object_utils(directed)
  // constructor
  ...
  virtual task body();
    `uvm_info(get_type_name(), "Call directed", UVM_LOW)
    `uvm_create(req)
    ok = req.randomize();
    req.addr = 3;
    req.set_parity();
    `uvm_send(req)
  endtask
endclass
```

- Create and send macros are ideal for defining directed or semi-directed stimulus

21 © Cadence Design Systems, Inc. All rights reserved.



Create and send macro combinations can be useful for directed or semi-directed stimulus.

For example:

- Create a data instance using ``uvm_create()`.
- Optionally randomize the instance to create legal data (here make sure the YAPP packet length and payload are consistent).
- Overwrite the required fields of the instance (here to set the address).
- Perform any housekeeping on the data (here update parity).
- Send the instance without further randomization using ``uvm_send()`.

Sequence Properties

```
// 2 yapp packets to consecutive addresses
class yapp_seq_addr extends uvm_sequence #(yapp_packet);

    `uvm_object_utils(yapp_seq_addr)

    function new (string name = "yapp_seq_addr");
        super.new(name);
    endfunction

    rand bit[1:0] seqadd;
    constraint c1 { seqadd <= 2'b01; }

    task body();
        `uvm_info(get_type_name(), "Call yapp_seq_addr", UVM_LOW)
        `uvm_do_with( req, {addr == seqadd;} )
        `uvm_do_with( req, {addr == seqadd+1;} )
    endtask

endclass
```

rand sequence property randomized
each time sequence called

Constraint for property randomization

Property used as constraints for packet generation

22 © Cadence Design Systems, Inc. All rights reserved.

cadence

Note that a sequence can have properties (e.g., `seqadd`). Properties can be used in the definition of the sequence behaviour. If the sequence property is declared as `rand`, then the property is randomized when the sequence is executed. The property can then be used anywhere in the `body()` task to generate stimulus, for example:

- In a repeat loop to create a variable number of packets.
- In `do_with` constraint blocks to define randomization constraints.

Constraints can be declared either in the sequence or in-line with the sequence call, to control the randomization of the sequence property. Here for example, we add a constraint to prevent `seqaddr` being 2 or 3, which would give us a sequence containing the illegal packet address 3.

Nesting Sequences

```
// call yapp_seq_addr sequence wrapped with random packets
class nested extends uvm_sequence #(yapp_packet);

    `uvm_object_utils(nested)
    // constructor

    yapp_seq_addr ysa;

    task body (); // sequence behavior
        `uvm_info(get_type_name(), "Call nested", UVM_LOW)

        `uvm_do(req)
        `uvm_do_with(ysa, {seqadd == 0;} ) // sub-sequence
        `uvm_do(req)

    endtask
endclass
```

Handle for previously defined sequence
Choose meaningful name for debug!

Constraint on yapp_seq_addr
sequence rand property

23 © Cadence Design Systems, Inc. All rights reserved.

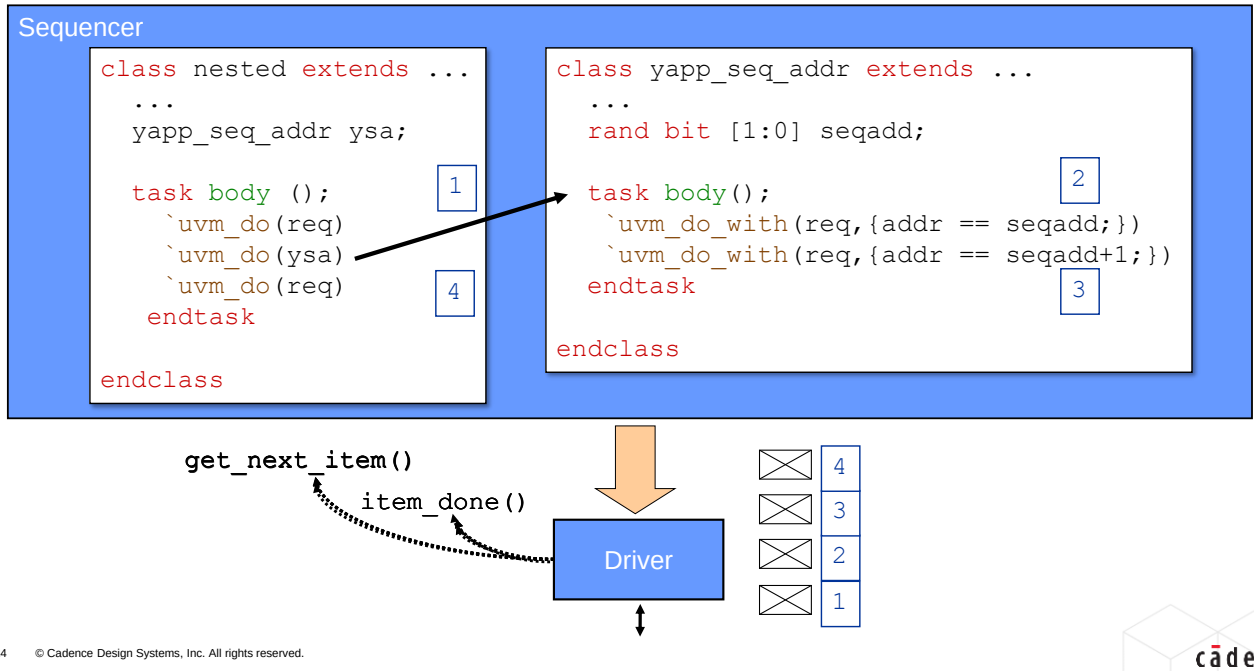


Sequences can be nested, for example to build up more complex stimulus from basic sequences.

A handle must be defined for the nested sequence, must be previously declared. It's a good idea to choose a meaningful name for the handle, related to the nested sequence, as this can help in debug (where only the handle name may be shown, not the nested sequence type).

A `uvm_do` macro can then be called on the sequence handle to execute it as a subsequence. The `uvm_do_with` macro variant can be used to pass inline constraints to the properties of the subsequence.

Sequence Execution During Simulation Run Time

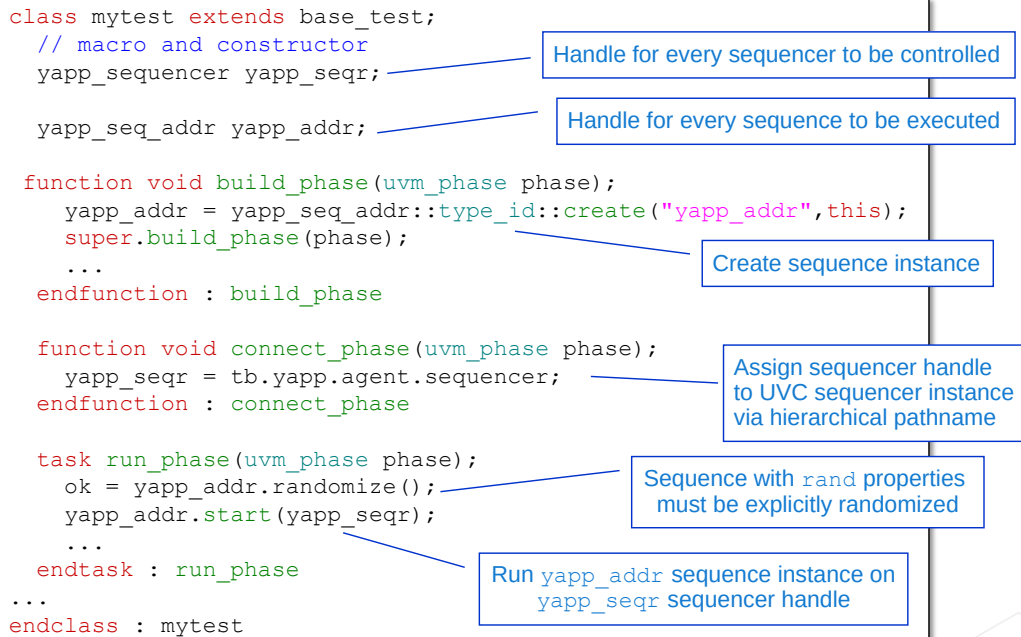


24 © Cadence Design Systems, Inc. All rights reserved.



This page does not contain notes.

Executing UVC Sequence in Test



25

© Cadence Design Systems, Inc. All rights reserved.

cadence

As an alternative to setting the default sequence of a sequencer, a test class may execute sequences directly on a sequencer.

The procedure is as follows:

- Declare a handle for the sequence(s) to be executed and create all these handles in the build phase of the test.
- Declare a handle for the UVC sequencer(s) on which the sequences will be executed. These handles are not created, but will be linked directly to the UVC sequencer instances.
- In the connect phase of the test, assign the path name of the UVC sequencer instance to the sequencer handle. The pathname must be relative to the test instance and use the handle names (not the instance names) of the components in the path. Wildcards are not allowed in the pathname – we are making a direct reference copy of the sequencer instance, not making a configuration setting.
- If the sequence instance contains rand properties, then the sequence must be explicitly randomized to give values to these properties. If you do not randomize the sequence, rand properties are left at their default value. Randomization could be done once in the build_phase, after the instance creation, or many times in the run_phase, as required.
- In the run phase of the test, execute the sequence instance on the sequencer reference using start.

Reading a UVC Sequencer Pathname from a Topology Report

xrun.log

```
...
UVM_INFO @ 0: reporter [UVMTOP] UVM testbench topology:
-----
```

Name	Type	Size	Value
uvm_test_top	base_test	-	@2629
tb	router_tb	-	@2696
yapp	yapp_env	-	@2742
tx_agent	yapp_agent	-	@2769
driver	yapp_driver	-	@3476
...			
monitor	yapp_monitor	-	@2805
sequencer	yapp_sequencer	-	@2801
...			
is_active	uvm_active_passive_enum	1	UVM_ACTIVE

UVC appear
under testbench

Instance names

Test executed



Question

What is the hierarchical pathname to the YAPP sequencer from the test class?

26 © Cadence Design Systems, Inc. All rights reserved.

cadence

A topology report provides a huge amount of useful information about your verification environment.

In Cadence's Xcelium simulator, the simulation output, including the topology report, is saved to the file `xrun.log`.

The topology report can tell you:

- What test is running (type of `uvm_test_top`). Is this correct?
- The hierarchy of your test environment. Is every UVC present and is every UVC structure as expected?
- Starting value of key properties (e.g., `is_active`). Is every property printed? If not, you may have forgotten to automate the property to enable printing.

A topology print is a useful debug feature for an UVM testbench. It allows the designer to check the hierarchy structure and property values of the testbench, and is key in understanding which test is running; which agents are present and which sequences are running. Always include this feature somewhere in your testbench.

Ending Simulation

An objection (consensus) mechanism is used to elegantly end the simulation.

- Simulation immediately tries to end the run phase
- A sequence must raise an objection when it starts to hold off the phase end
- A sequence must drop the objection on finishing to allow the phase to end
- After a certain time (drain time) with no further objections raised, the phase ends

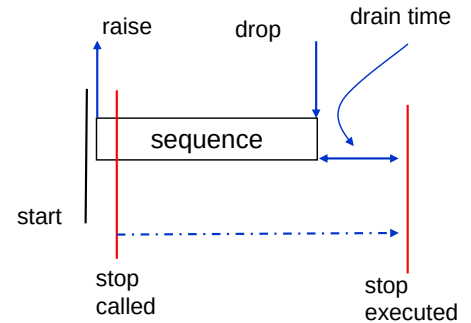
Every run (sub)phase has a built-in objection mechanism.

The run phase ends if no objections were raised or when all objections are removed.

- Expandable, reusable, and flexible

Not all sequences should raise objections

- Not appropriate for background traffic, response generators, or subsequences



The objection mechanism manages the termination of `run_phase` and hence simulation end.

The `run_phase` has a built-in objection mechanism. In addition, every run sub-phase also has its own objection handling.

At the start of every run phase, the objection mechanism tries to end the phase immediately. If a sequence executes in that phase, it must raise an objection to the phase end. Raised objections prevent the phase ending until dropped. Once the sequence has ended, it must drop the objection.

If no objections are raised for a phase, or when the objection count returns to zero, then the phase waits for a certain period of time (called the drain time) in anticipation of another objection being raised. If no new objections are raised during the drain time, the phase ends. The default drain time is zero, i.e., the phase will end immediately when the objection count reaches zero or if no objections are raised.

When all run sub-phases have ended, and the `run_phase` itself has ended, then the simulation progresses to `extract_phase`, `check_phase` and `report_phase`.

Only `run_phase` and its sub-phases can rise objections as these are the only phases which consume time.

For efficiency and robustness, you should raise the minimum number of objections to maintain the simulation. Therefore not all sequences should handle objections. For example background, response, or nested sequences.

Test Objections

Objections are raised and dropped by methods

```
raise_objection(<object>, <description>, <count>);
drop_objection(<object>, <description>, <count>);
```

- Raise or drop the number of objections for object by count (default 1)
- description is a string used for tracing and debug

In a test, objection methods are called in the run phase (or subphase)

- Using the phase method argument

```
class mytest extends uvm_test;
...
task run_phase(uvm_phase phase);
    phase.raise_objection(this, "mytest");

    yapp012.start(yapp_seqr);
    ...
    phase.drop_objection(this, "mytest");
endtask : run_phase
...
```

Raise an objection
for this component
for run phase

Drop objection
after stimulus sent

28 © Cadence Design Systems, Inc. All rights reserved.

cadence

Objections are raised and dropped via methods:

virtual function void raise_objection (uvm_object obj = null,
string description = "", int count = 1)

Raises the objection count for the nominated object (use this to specify the calling object) by count. The description string argument is used for debug.

virtual function void drop_objection (uvm_object obj = null,
string description = "", int count = 1)

Drops the objection count for the nominated object (use this to specify the calling object) by count. The description string argument is used for debug. Dropping the objection count below 0 is an error.

In a test class, objections are raised and dropped at the beginning and end of the run phase method and called from the phase argument of the method.

Objections propagate up the hierarchy to uvm_top so that the simulation can keep track of objections raised by every UVC. Once the uvm_top objection count for a phase returns to zero (or if no objections are raised for a phase) the phase ends.

Summary

This was only a very brief introduction to UVM.

We covered:

- The basic building blocks of a UVM test environment
 - Interface and module UVCs
- The structure of a UVM test environment
- How to write and execute UVC stimulus using sequences

UVM makes writing stimulus for an existing environment relatively easy

- No knowledge of DUT protocols required
- Minimal knowledge of the structure of a UVM testbench

Building a UVM environment is a much more difficult task

- This is covered in detail in the full Cadence UVM training class



This page does not contain notes.